

Final Project – SQL Part

בשלב הראשונים בנינו בסיס מקצועי של מחסן נתונים (DWH) שמוכן להריץ עליו ניתוחים, דוחות, ותרשימים שונים בצורה הכי מסודרת על ידי בניית Dimension Tables ו Fact Tables ועל פי המודל המקובל - Star Schema.

Dimension Tables

- **Dim_playlist**
טבלה שמקשרת בין פלייליסטים לשירים בעזרת INNER JOIN על מנת להביא רק שירים ששויכו לפלייליסטים קיימים בפועל, ולסנן החוצה פלייליסטים או שירים שאין להם התאמות. שימוש ב- INNER JOIN מבטיח שהנתונים שנקבל הם רק חיבורים אמיתיים בין פלייליסטים לשירים, בדיוק לפי דרישות בניית טבלת Dim_playlist.
- **Dim_customer**
טבלת לקוחות נקייה: ניקינו שמות, חילצנו דומיינים מאימיילים ושמרנו מידע אישי ותקשורתי. מאפשרת פילוחים וניתוחים לפי מאפייני לקוחות.
- **Dim_employee**
טבלת עובדים, כולל מידע על מחלקות, ותק, זיהוי מנהלים ודומיין אימייל. מאפשרת ניתוח לפי עובדים, מחלקות ומנהלים.
- **Dim_track**
טבלה שמרכזת את כל המידע על שירים – כולל אמן, אלבום, סוג מדיה, ז'אנר ואורך שיר בפורמט קריא. מאפשרת ניתוח לפי סוגי מוזיקה, אמנים ואלבומים.

Fact Tables

- **Fact_invoice**
מזהה לקוח, תאריך החשבונית והסכום הכולל. מאפשרת ניתוח של מכירות לפי לקוחות, תקופות וזמנים.
- **Fact_invoiceline**
מפורט על כל שיר שנמכר, כולל חישוב סך התשלום לכל שורת קנייה. מאפשרת ניתוח מפורט לפי שירים ומוצרים שנמכרו.

```

DO $$
DECLARE
    rec RECORD;
BEGIN
    -- Find all tables in the 'stg' schema
    FOR rec IN
        SELECT table_name
        FROM information_schema.tables
        WHERE table_schema = 'stg'
    LOOP
        -- For each table, move it to the new 'dwh' schema
        EXECUTE 'ALTER TABLE stg.' || rec.table_name || ' SET SCHEMA dwh';
    END LOOP;
END $$;

ALTER TABLE stg."dim_currency" SET SCHEMA dwh;

```

בשלב זה ביצענו העברת טבלאות מסכמת העבודה stg אל סכמת מחסן הנתונים dwh כדי לרכז את כל הטבלאות שאנו עובדים איתן תחת סכמה אחת ובכך לשמור על הסדר. השתמשנו בפקודת DO \$\$ DECLARE BEGIN ... END \$\$ כדי להריץ בלוק אנונימי ב-PostgreSQL שמבצע לולאה. בתוך הלולאה שלפנו את כל שמות הטבלאות הקיימות תחת הסכמה stg מתוך טבלת information_schema.tables. עבור כל טבלה שמצאנו ביצענו ALTER TABLE דינמי כדי להעביר אותה לסכמה החדשה dwh. בנוסף, הרצנו ידנית פקודת ALTER TABLE עבור טבלה אחת ספציפית – dim_currency להעבירה גם היא לסכמה dwh.

Data Analysis in SQL

Question 1

בשלב זה ביצענו ניתוח על כמות השירים בכל פלייליסט. יצרנו תחילה טבלה זמנית (playlist_counts) שמחשבת כמה שירים יש לכל פלייליסט. לאחר מכן יצרנו טבלה זמנית נוספת (playlist_with_stats) שמוסיפה נתונים סטטיסטיים לכל פלייליסט: מה מספר השירים המקסימלי (MAX(total_tracks)) מה מספר השירים המינימלי (MIN(total_tracks)) מה ממוצע השירים לכל פלייליסט (AVG(total_tracks)) בשאלתה הסופית: הצגנו רק את הפלייליסטים שיש להם את מספר השירים הגבוה ביותר או הנמוך ביותר. לכל פלייליסט שסיגנו הצגנו:

- האם הוא פלייליסט עם הכי הרבה שירים (Max Tracks) או עם הכי מעט שירים (Min Tracks).
- מזהה הפלייליסט.
- מספר השירים שבו.
- הממוצע הכללי של מספר השירים בכל הפלייליסטים.

Question 2

בשלב זה ביצענו ניתוח כדי לקבץ את השירים לפי מספר המכירות שלהם. יצרנו טבלה זמנית (WITH) שמחשבת לכל שיר (trackid) את מספר הפעמים שהוא נמכר (sales_count) מתוך טבלת fact_invoice_line. הקוד מנתח את מספר המכירות לכל שיר ומחלק את השירים לקבוצות לפי טווחי מכירה: 0 מכירות, 1-5 מכירות, 5-10 מכירות, 10-50 מכירות ו-100+ מכירות ומעלה. זה מאפשר להבין כמה שירים נמכרים הרבה, וכמה שירים לא נמכרים כמעט בכלל.

Question 3

-3a

בשלב זה ביצענו ניתוח מכירות לפי מדינה. יצרנו תחילה טבלה זמנית sum_sales_per_country שבה חישבנו את סך כל המכירות לכל מדינה יצרנו שתי טבלאות זמניות נוספות:

- top_5_countries חמש המדינות עם המכירות הכי גבוהות.
- bottom_5_countries חמש המדינות עם המכירות הכי נמוכות.

אחר כך איחדנו (UNION ALL) את התוצאות משתי הקבוצות לטבלה אחת (selected_countries). לבסוף, שלפנו את כל המדינות שנבחרו, ממוינות לפי סך המכירות מהגבוהה לנמוך. הקוד מחשב סך מכירות לפי מדינה, מאתר את חמש המדינות שמובילות ואת חמש המדינות שמפגרות במכירות, ומציג אותן בסדר יורד של סך המכירות.

-3b

בשלב זה ביצענו ניתוח של מכירות לפי ז'אנר מוזיקה בתוך המדינות הכי חזקות והכי חלשות במכירות. תחילה חישבנו (sum_sales_per_country) את סך כל המכירות לכל מדינה. בחרנו את חמש המדינות עם הכי הרבה מכירות (top_5_countries) ואת חמש המדינות עם הכי מעט מכירות (bottom_5_countries). איחדנו את המדינות האלו לרשימה (selected_countries).

לאחר מכן חישבנו (genre_sales_per_country) את סך המכירות לכל ז'אנר מוזיקה בכל אחת מהמדינות שנבחרו.

בשאלתה הסופית:

- חיברנו את סך המכירות של כל ז'אנר למדינה הכוללת.
- חישבנו את אחוז המכירות של כל ז'אנר מתוך סך המכירות של אותה מדינה.
- השתמשנו בפונקציה RANK() כדי לדרג את הז'אנרים לפי האחוז במדינה (מהנמכר ביותר לפחות).

הקוד מחשב את תרומת כל ז'אנר לסך המכירות במדינות מובילות וחלשות, ומדרג את הז'אנרים הפופולריים ביותר בכל מדינה, לפי אחוז מהמכירות הכוללות.

Question 4

בשלב זה ניתחנו את פעילות הלקוחות בכל מדינה תוך חישוב ממוצעים חשובים.

בשלב ראשון customer_summary חישבנו לכל מדינה:

- כמה לקוחות יש num_customers
- כמה חשבוניות יש total_invoices
- מה סך ההכנסות מאותה מדינה total_revenue

לאחר מכן customer_with_group

- אם במדינה יש רק לקוח אחד קיבצנו אותה תחת קטגוריה בשם 'Other', שאר המדינות נשארו בשמות שלהן.

בשלב הבא country_grouped

- סיכמנו את מספר הלקוחות, החשבוניות וההכנסות לכל קבוצה.

ובשאלתה הסופית חישבנו:

- את ממוצע החשבוניות לכל לקוח
- את ממוצע ההכנסה מכל לקוח
- מיינו כך שהקבוצות האמיתיות (מדינות) יוצגו קודם וקטגוריית 'Other' תוצג בסוף.

הקוד מנתח את מספר הלקוחות, ממוצע מספר ההזמנות וממוצע ההכנסה לכל לקוח לפי מדינה, תוך קיבוץ מדינות קטנות (לקוח אחד בלבד) לקטגוריית 'Other'.

Question 5

בשלב זה בנינו ניתוח שמרכז את ביצועי המכירות של כל עובד, כולל חישוב אחוז הצמיחה משנה לשנה.

בשלב הראשון employee_base יצרנו טבלה בסיסית שמציגה לכל עובד:

- מזהה עובד
- שם מלא
- שנת התחלת עבודה
- ותק בשנים

בשלב השני sales_per_employee_per_year

- חיברנו לקוחות למנהלי החשבון שלהם (העובדים)
- חישבנו לכל עובד בכל שנה כמה לקוחות טיפל ומה סך המכירות שטיפל בהן.

בשלב השלישי growth_calculation

- חישובנו את סך המכירות של כל עובד בכל שנה
- חילצנו את סך המכירות של השנה הקודמת באמצעות פונקציית חלון LAG().

בשאלתה הסופית:

- הצגנו את הנתונים לכל עובד, לכל שנה,
- ובנוסף חישובנו את אחוז הצמיחה במכירות משנה לשנה, אם אין שנה קודמת או המכירות היו אפס — הצגנו 0%.

הקוד מחשב מכירות שנתיות ואחוז צמיחה לכל עובד, תוך שילוב מידע על שם העובד, שנות ותק, ומספר הלקוחות שטיפל בהם כל שנה.

```
SELECT
    dc.Country,
    COUNT(DISTINCT dc.CustomerId) AS Num_Customers,
    COUNT(fi.InvoiceId) AS Num_Invoices,
    SUM(fi.Total) AS Total_Revenue,
    SUM(fi.Total) / COUNT(DISTINCT dc.CustomerId) AS Revenue_Per_Customer,
    AVG(fi.Total) AS Avg_Invoice_Amount
FROM
    Dim_customer AS dc
JOIN
    fact_invoice AS fi ON dc.customerid = fi.customerid
GROUP BY
    dc.Country
ORDER BY
    Revenue_Per_Customer DESC
LIMIT 5;
```

ניתוח הכנסה ללקוח לפי מדינות – זיהוי הזדמנויות שיווקיות ואסטרטגיות

בשלב זה ביצענו שאלתה שמטרתה לזהות את חמש המדינות שבהן ההכנסה ללקוח היא הגבוהה ביותר. החישוב בוצע על ידי חיבור בין לקוחות לחשבונות, סיכום כלל המכירות, וחשוב ההכנסה הממוצעת ללקוח במדינה. מיון התוצאות לפי הכנסה ללקוח איפשר לנו לזהות מדינות איכותיות – כאלה שמניבות הכנסה גבוהה יותר לכל לקוח בודד.

מסקנות הניתוח תומכות בקבלת החלטות אסטרטגיות:

- איתור הזדמנויות לשימור והגברת פעילות במדינות מובילות.
- אפיון קהלי יעד איכותיים לשיווק ממוקד.
- השוואה למדינות חלשות לקבלת תמונה רחבה של ביצועים.
- בדיקת פוטנציאל התרחבות חכמה לשווקים רווחיים.

Final Project – Python Part

ייבוא ספריות וטעינת קבצי מקור

בשלב הפתיחה של הפרויקט ייבאנו את הספריות המרכזיות: pandas, numpy ו- json המשמשות לניתוח, עיבוד וטעינה של נתונים. מיד לאחר מכן, טענו ארבעה קבצים חיצוניים מסוגים שונים לתוך סביבת העבודה, תוך התאמה של כל פורמט לקוד הקריאה המתאים לו. כל קובץ הומר לטבלת DataFrame נפרדת, כהכנה לשלבי מיזוג וניתוח בהמשך.

איחוד נתוני תקציב ממקורות שונים

בשלב זה איחדנו את שני קובצי התקציב שהתקבלו department_budget1_df ו- department_budget2_df לטבלה אחת מאוחדת בשם department_budget_df. האיחוד בוצע באמצעות הפונקציה pd.concat על ציר השורות כך שכל השורות משני המקורות חוברו זו תחת זו.

מיזוג טבלאות וחישוב תקציב כולל למחלקה

בשלב הזה מיזגנו בין טבלת המחלקות (department_df) לבין טבלת התקציבים המאוחדת department_budget_df. לאחר המיזוג, קיבצנו את הנתונים לפי מזהה ושם מחלקה, וחישבנו את סך התקציב הכולל לכל מחלקה. מטרת הפעולה היא ליצור טבלה מרוכזת המציגה עבור כל מחלקה את התקציב הכולל שלה בצורה ברורה וניתנת להמשך ניתוח.

טעינת טבלה למסד PostgreSQL

והעלינו אליו את טבלת המיזוג PostgreSQL בשלב זה חיברנו את סביבת העבודה למסד הנתונים merged_df. תחילה הוגדרו פרטי ההתחברות למסד ולאחר מכן נבנה engine המייצג את החיבור בפועל. לבסוף השתמשנו בפונקציה to_sql כדי להעלות את הטבלה לסכימה stg תוך שימוש בפרמטר if_exists='replace' המחליף טבלה קיימת באותו שם במידת הצורך. השלב מסתיים בהודעת אישור שמעידה על הצלחה בהעלאת הטבלה למסד.

API - שליפת שערי המרה לפי תאריכים ושמירתם

בשלב זה שלפנו שערי המרה בין דולר לשקל לפי תאריכי העסקאות בעזרת שימוש ב-API של Open Exchange Rates. התחלנו מהמרת עמודת התאריכים לפורמט מתאים לשליפה, והשתמשנו בלולאת for כדי לבצע בקשה נפרדת לכל תאריך. כל תוצאה מוצלחת הוספה לרשימת שערים, ובמקרה של תקלה התקבלה הודעת שגיאה. לאחר השליפה, הוספנו את עמודת ההמרה usd_to_ils ל- DataFrame המקורי transactions_dates_df ושמרנו את הקובץ המעודכן כקובץ אקסל חדש להמשך שימוש. שלב זה מאפשר לנו לשלב מידע פיננסי חיצוני חשוב בנתונים, כהכנה לניתוחים כלכליים עמוקים יותר בהמשך.

טעינת טבלת תאריכי העסקאות למסד הנתונים

בשלב זה העלינו את טבלת transactions_dates_df הכוללת את נתוני העסקאות בצירוף שערי ההמרה, אל מסד הנתונים PostgreSQL. בוצעה התחברות למאגר בעזרת פרטי ההזדהות וה- host שסופקו מראש, באמצעות ספריית sqlalchemy לאחר יצירת החיבור, השתמשנו ב- to_sql להעלאת הטבלה לסכימה stg תוך מחיקת טבלה קודמת באותו שם אם קיימת (if_exists='replace') שלב זה מאפשר גישה נוחה לנתונים המעודכנים ישירות מהמסד, כהכנה לשליפת מידע וניתוחים בשלב הבא.

טעינת טבלאות מחסן הנתונים אל Google Colab

בשלב זה יצרנו חיבור בין סביבת Google Colab לבין מסד הנתונים PostgreSQL וייבאנו ממנו את טבלאות העבודה המרכזיות.

לאחר ייבוא ספריות חיוניות כמו pandas, sqlalchemy ו-matplotlib הוגדר מנגנון חיבור (engine) באמצעות פרטי ההתחברות למסד.

בהמשך, נשלפו באמצעות שאילתות SQL כל טבלאות הדאטה המרכזיות שנבנו בשלבים הקודמים, מתוך הסכימה dwh כל טבלה נשמרה כ- DataFrame נפרד בסביבת העבודה. פעולה זו מאפשרת לבצע אנליזות, גרפים וטיוב נתונים בצורה נוחה וישירה מתוך Colab.

Data Analysis in Python

Question 1

בשלב זה ביצענו ניתוח על בסיס טבלת השירים dim_track במטרה להבין אילו אמנים וז'אנרים בולטים במסד הנתונים שלנו.

התחלנו מהפקת שלוש טבלאות שמרכזות תובנות שונות:

- הצגת חמשת האמנים עם מספר האלבומים הגבוה ביותר (top_5_artists_with_albums)
- הצגת חמשת האמנים עם מספר השירים הרב ביותר (top_5_artists_with_tracks)
- הצגת חמשת הז'אנרים הפופולריים ביותר לפי כמות שירים (top_5_genres_with_tracks)

לאחר שאיחדנו את הנתונים האלו, יצרנו שלוש תצוגות גרפיות מסוג bar chart, כשכל אחת מהן ממחישה את חלוקת הכמות בין ישויות שונות (אמן או ז'אנר).

שלושת התרשימים שובצו יחד באותה פלטת תצוגה (subplot) בעזרת ספריית matplotlib תוך שימוש בצבעים ייחודיים לכל גרף ובציר X ברור שמייצג את שמות האמנים או הז'אנרים.

שלב זה נועד לספק מבט על מבנה תוכן המוזיקה הקיים: אילו אמנים אחראים על הנפח המשמעותי ביותר של מוזיקה, ואילו ז'אנרים שולטים בפלטפורמה.

Question 2

בשלב זה ביצענו ניתוח להשוואת סך הרכישות של הלקוחות המובילים בשני מטבעות – דולר (USD) ושקל (ILS).

תחילה ביצענו התאמה בין טבלת שערי ההמרה dim_currency לבין טבלת החשבוניות fact_invoice על ידי שינוי שם העמודה Dates ל- invoicedate כדי לאפשר מיזוג לפי תאריך.

לאחר המיזוג, יצרנו עמודה חדשה ils_amount על ידי הכפלת סכום הרכישה בדולר (total) בשער ההמרה התואם לאותו יום (usd_to_ils) במקביל שמרנו את הערך המקורי בעמודת usd_amount.

כדי להבין מי מהלקוחות ביצע את הרכישות הגבוהות ביותר, מיזגנו את הנתונים עם טבלת הלקוחות

dim_customer ולאחר מכן קיבצנו את הנתונים לפי מזהה לקוח וחישבנו את סכום הרכישות המצטבר בשני המטבעות.

סיננו את חמשת הלקוחות בעלי הסכום הגבוה ביותר בשקלים.

כדי להציג את המידע בצורה ברורה, המרנו את הטבלה לפורמט ארוך (long format) בעזרת pd.melt שאיפשר להבחין בין שני המטבעות בעמודת Currency את התוצאה הצגנו בגרף עמודות מרובה שכבות (catplot) עם חלוקה לפי מטבע בצבעים שונים.

גרף זה מציג את ההשוואה בין הרכישות של כל לקוח במונחי דולר ושקל, וממחיש אילו לקוחות אחראים להכנסה הגבוהה ביותר לאחר המרת המטבע.

Question 3

בשלב זה ביצענו ניתוח זמן של היקף המכירות, במטרה לזהות מגמות לאורך השנים ברמה חודשית. ראשית, חילצנו מתוך עמודת invoicedate את השנה והחודש של כל רכישה, ויצרנו שדות חדשים בטבלה לצורך קיבוץ.

לאחר מכן קיבצנו את הנתונים בטבלת fact_invoice לפי שנה וחודש, וחישבנו את סך המכירות בכל קבוצה. כדי להבטיח סדר כרונולוגי נכון להצגה, מיינו את התוצאה לפי שנה וחודש.

את רשימת השנים הקיימות שלפנו אוטומטית מתוך הטבלה כדי לוודא שהקוד יהיה דינמי ויתאים גם לנתונים חדשים.

לאחר מכן בנינו גרף עמודות עבור כל שנה בנפרד באמצעות matplotlib כאשר כל גרף מציג את התפלגות המכירות לפי חודשי השנה הרלוונטית.

המטרה של ניתוח זה היא לזהות תקופות שיא או ירידה בהכנסות לאורך הזמן, להבין עונתיות אפשרית בהתנהגות הלקוחות, ולזהות דפוסים חוזרים.

Question 4

בשלב זה בחנו האם קיים קשר בין אורך השיר לבין סך המכירות שלו. תחילה ביצענו מיזוג בין טבלת השירים `dim_track` לבין טבלת שורות החשבוניות `fact_invoiceline` במטרה לצרף לכל שיר את נתוני המכירות שלו. לאחר המיזוג, קיבצנו את הנתונים לפי מזהה שיר (`trackid`) ואורך השיר (`length`) וחישבנו את סכום הרכישות הכולל (`line_total`) לכל שיר. בהמשך סיננו את עשרת השירים בעלי המכירות הגבוהות ביותר להצגה, במטרה להתמקד בשירים הפופולריים והרלוונטיים ביותר לניתוח. את המידע הצגנו באמצעות גרף פיזור (`scatter plot`) הממחיש את הקשר בין אורך השיר לבין סך המכירות. בנוסף, סומנו בצבע ייחודי (אדום) נקודות מסוימות לפי אורכים בולטים שנבחרו מתוך התוצאה, כדי להדגיש ולזהות דפוסים חוזרים. המסקנה מהגרף היא שנראה כי קיים קשר בין אורך השיר לרמת המכירות – שירים באורך של כ-43 דקות מופיעים במספר מקרים ברשימת השירים המובילים מבחינת הכנסה. ניתוח זה מאפשר להבין האם יש קשר בין מאפיינים טכניים של שיר (כמו משך הזמן שלו) לבין ההצלחה הכלכלית שלו, ויכול להוות בסיס לשיקולים ביצירת תכנים או בתמחור שירים בפלטפורמה.

Question 5

בשלב זה פיתחנו פונקציית המלצה דינמית, שמטרתה להציע ללקוח שירים חדשים המבוססים על תחומי העניין האישיים שלו. הפונקציה `recommend_tracks_for_customer` מקבלת מספר מזהה לקוח (`customerid`) ומבצעת מספר שלבים לוגיים: תחילה נבדק האם הלקוח קיים במערכת וכי קיימות עבורו רכישות. לאחר מכן מאוחדים נתוני החשבוניות, שורות החשבוניות ופרטי השירים, כדי ליצור תמונה מלאה של התנהגות הקנייה של הלקוח. בהמשך, חושבו שני הז'אנרים האהובים ביותר על הלקוח על בסיס כמות הרכישות, וזוהו השירים בז'אנרים אלו שהלקוח עדיין לא רכש. עבור כל אחד מהז'אנרים נבחרו שלושת השירים הנמכרים ביותר (שלא נרכשו על ידי הלקוח) בהתבסס על כמות הרכישות הכללית. התוצאה הסופית מוצגת בטבלה שמכילה את מזהה הלקוח, מזהה השיר, שם השיר, שם הז'אנר וכמות הרכישות – ומאפשרת להציג ללקוח המלצות מותאמות אישית ואיכותיות. הפונקציה מאפשרת להרחיב את חוויית המשתמש באופן חכם, מגדילה את הפוטנציאל למכירות נוספות, ומבוססת על פרופיל ההתנהגות הקיים של כל לקוח.

ניתוח מגמת מכירות לפי רבעונים

בשלב זה ביצענו ניתוח מגמה של המכירות ברמה רבעונית, על מנת לזהות שינויים והתנהגויות לאורך זמן. ראשית, המרה עמודת התאריכים `invoicedate` לפורמט תאריך תקני אפשרה לנו לחלץ ממנה את שנת הרכישה ואת הרבעון המתאים (`quarter`) כאשר הרבעון מיוצג בפורמט משולב (לדוגמה: Q1.2024). לאחר מכן קיבצנו את טבלת `fact_invoice` לפי רבעון וחישבנו את סך המכירות בכל רבעון. את התוצאה הצגנו באמצעות גרף קו, המדגיש את מגמת המכירות לאורך התקופות, ומאפשר לזהות מגמות עונתיות, נקודות עלייה או ירידה בהכנסות, ותנועת פעילות העסק לאורך זמן. ניתוח זה מספק מבט על התנהגות המכירות באופן תקופתי, ומהווה בסיס לתכנון אסטרטגי, ניהול מלאי, קמפיינים שיווקיים, וחיזוי עתידי של ביצועים.